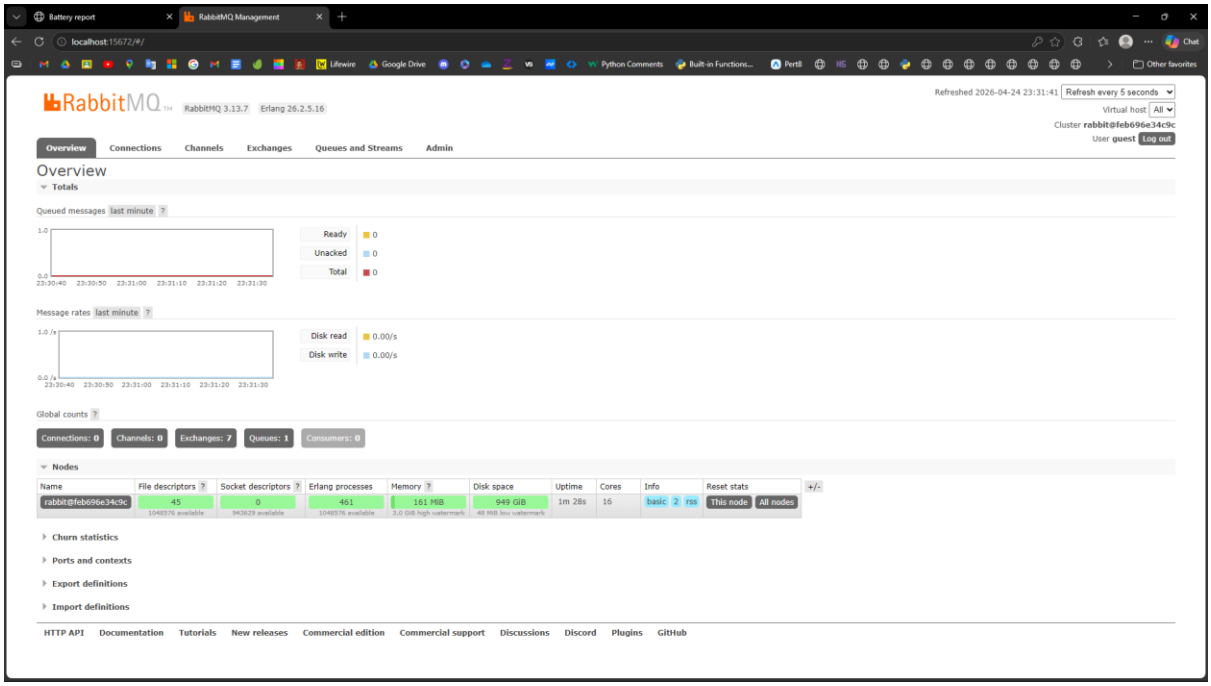
Nama: Joshua Taftfarel Petra Chrisensa
NPM: 232310002
Kelas: TI-23-KA

Tampilan Rabbit MQ yang dijalankan melalui docker



Menginstall rabbitmq pada program

Penambahan api pada .env untuk memanggil rabbit mq

```
RABBITMQ_URL=amqp://localhost:5672
```

Penambahan kode program config untuk rabbit my

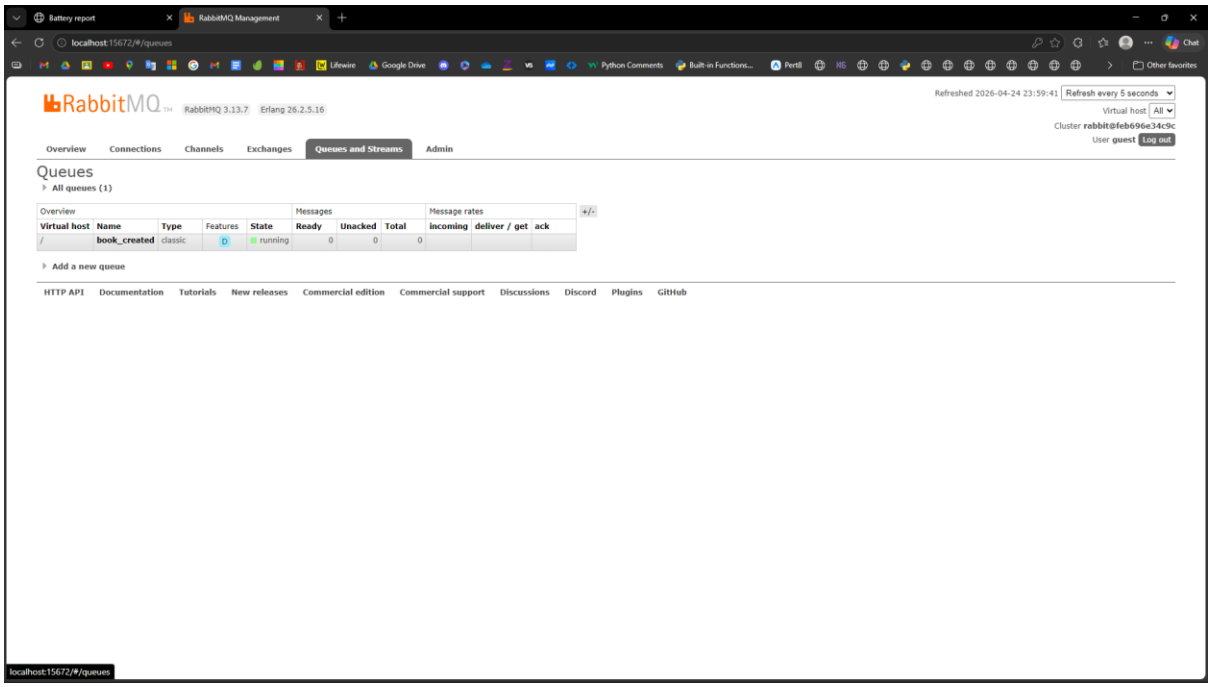
BookServices > config > rabbitmq.js > ...

```
const amqp = require("amqplib");
class RabbitMQService {
  constructor() {
    this.connection = null;
    this.channel = null;
  }
  async connect() {
    try {
      this.connection = await amqp.connect(
        process.env.RABBITMQ_URL || "amqp://localhost",
      );
      this.channel = await this.connection.createChannel();
      console.log("Connected to RabbitMQ");
      return true;
    } catch (error) {
      console.error("Failed to connect to RabbitMQ:", error);
      return false;
    }
  }
  async sendToQueue(queueName, message) {
    try {
      if (!this.channel) {
        await this.connect();
      }
      await this.channel.assertQueue(queueName, { durable: true });
      this.channel.sendToQueue(
        queueName,
        Buffer.from(JSON.stringify(message)),
        { persistent: true },
      );
      console.log(`Message sent to queue ${queueName}:`, message);
      return true;
    } catch (error) {
      console.error("Error sending message to queue:", error);
      return false;
    }
  }
  async close() {
    try {
      if (this.channel) await this.channel.close();
      if (this.connection) await this.connection.close();
      console.log("RabbitMQ connection closed");
    } catch (error) {
      console.error("Error closing RabbitMQ connection:", error);
    }
  }
}
module.exports = new RabbitMQService();
```

Penambahan await rabbitmq untuk mengirimkan log penambahan pada function createBook

```
exports.createBook = async (req, res) => {
  try {
    const {
      title,
      author,
      rating,
      views,
      is_free,
      language,
      sinopsis,
      story,
      image,
    } = req.body;
    // Validation
    if (!title || !author || !sinopsis || !story) {
      return res.status(400).json({
        success: false,
        message: "Title, author, sinopsis, and story are required",
      });
    }
    const book = await Book.create({
      title,
      author,
      rating: rating || 0,
      views: views || 0,
      is_free: is_free || false,
      language: language || "English",
      sinopsis,
      story,
      image,
    });

    // send data to message broker
    await rabbitmq.sendToQueue("book_created", {
      event: "book.created",
      timestamp: new Date().toISOString(),
      data: {
        id: book.id,
        title: book.title,
        author: book.author,
        is_free: book.is_free,
        language: book.language,
      },
    });
    //end of message broker

    res.status(201).json({
      success: true,
      message: "Book created successfully",
      data: book,
    });
  } catch (error) {
    console.error("Error creating book:", error);
    // Handle validation errors
    if (error.name === "SequelizeValidationError") {
      return res.status(400).json({
        success: false,
        message: "Validation error",
        errors: error.errors.map((e) => ({
          field: e.path,
          message: e.message,
        })),
      });
    }
    res.status(500).json({
      success: false,
      message: "Failed to create book",
      error: error.message,
    });
  }
};
```

Lokasi Melihat Log / get message pada rabbitmq setelah melakukan aksi create